

## Guia Prático 1

### Keil uVision, Flash and Debug (Aprendizagem das ferramentas)

Equipa Docente:

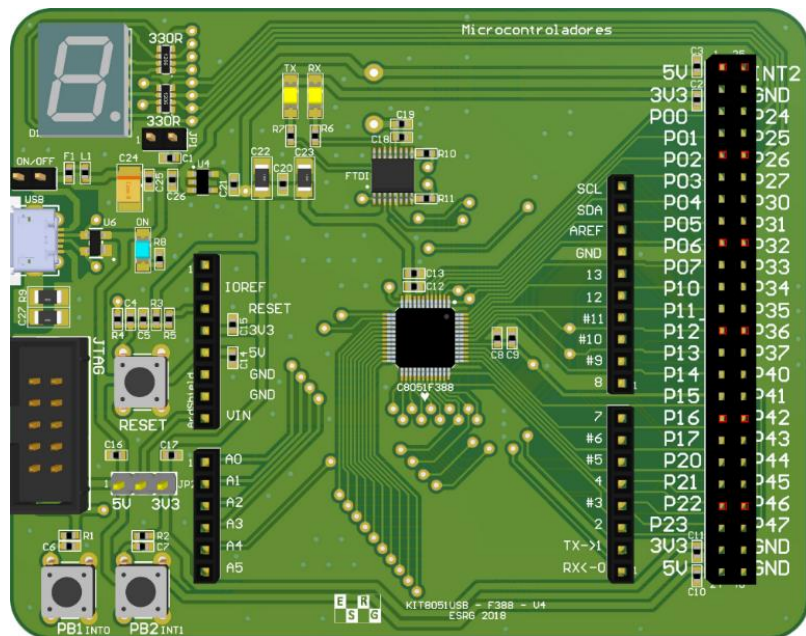
Tiago Gomes (PL)  
[mr.gomes@dei.uminho.pt](mailto:mr.gomes@dei.uminho.pt)

Paulo Cardoso  
[paulo.cardoso@dei.uminho.pt](mailto:paulo.cardoso@dei.uminho.pt)

Embedded Systems Research Group (ESRG)  
Departamento de Eletrónica Industrial (DEI) – Universidade Do Minho

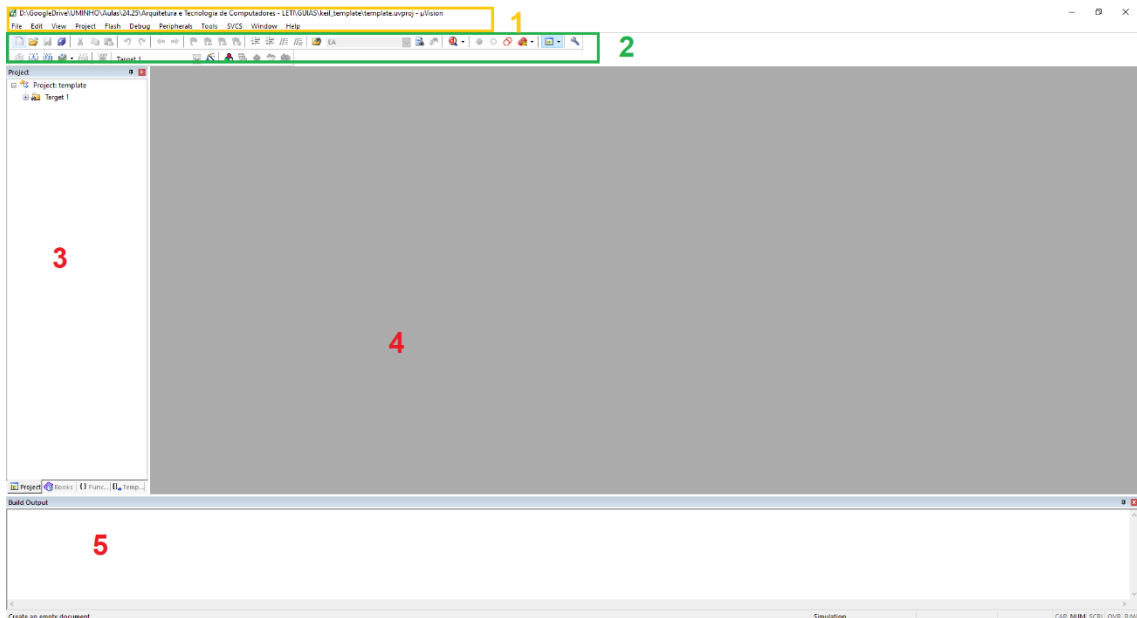
#### 1. Objetivos

- Conhecer a ferramenta de desenvolvimento Keil uVision;
- Criar e configurar um projeto para o microcontrolador C8051F388;
- Compilar o código e programar a placa;
- Aprender e realizar uma sessão completa de debug.



## 2. Silicon Keil Labs IDE

Comece por perceber a(s) ferramenta(s) e as funcionalidades que o Keil oferece.



1. Menus – menus de acesso a todas as funcionalidades do IDE;
2. Atalhos – atalhos para algumas funcionalidades do IDE;
3. Project View – janela com a hierarquia do projeto e com os ficheiros a compilar
4. Editor – janela principal para edição do código e outras fontes de texto;
5. Mensagens – mensagens com os resultados do processo de assemblagem, como por exemplo erros e warnings.

Crie um novo projeto no Keil IDE como aprendeu no Guia 0, e configure-o para utilizar o microcontrolador C8051F388 (ou o que tiver disponível), presente no KIT8051 disponibilizado.

Codifique o seguinte programa:

```
#include <REG51F380.H>

sbit seg_A = P2^7;

char msg1[] = {"STRING 1\n"};
char data msg2[] = {"STRING 2\n"};
char code msg3[] = {"STRING 3\n"};
char xdata msg4[] = {"STRING 4\n"};
char data msg5[] = {"STRING 5\n"};
char xdata msg6[10] _at_ 0x0100;
/*****/
void system_init(void){
    PCAOMD    = 0x00;
    FLSCL     = 0x90;
    CLKSEL    = 0x03;
    XBR1      = 0x40;
}
/*****/
void simple_delay(void){
    unsigned int i;
    for (i = 0; i < (unsigned int) 32768; i++);
}
/*****/
void main(){
    unsigned int counter = 0;      // A counter variable to observe
    unsigned char small_var = 0;  // A small variable to observe
    int signed_var = -10;         // A signed variable to observe
    float decimal_var = 0.5;      // A float variable to observe
    int i;

    system_init();

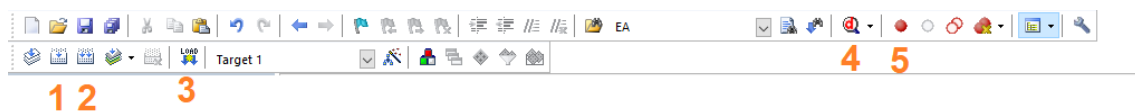
    for(i=0; msg3[i]!='\0'; i++) {
        msg6[i]=msg3[i];
    }
    msg6[i]='\0';

    while (1){
        seg_A ^= 1;

        while(counter++ < 10)    // Increment the counter
            small_var += 2;      // Increment the small variable by 2
        signed_var--;            // Decrement the signed variable
        decimal_var *= 1.1;      // Multiply the float by 1.1

        simple_delay();
    }
}
```

Compile o programa utilizando o botão **Rebuild**.



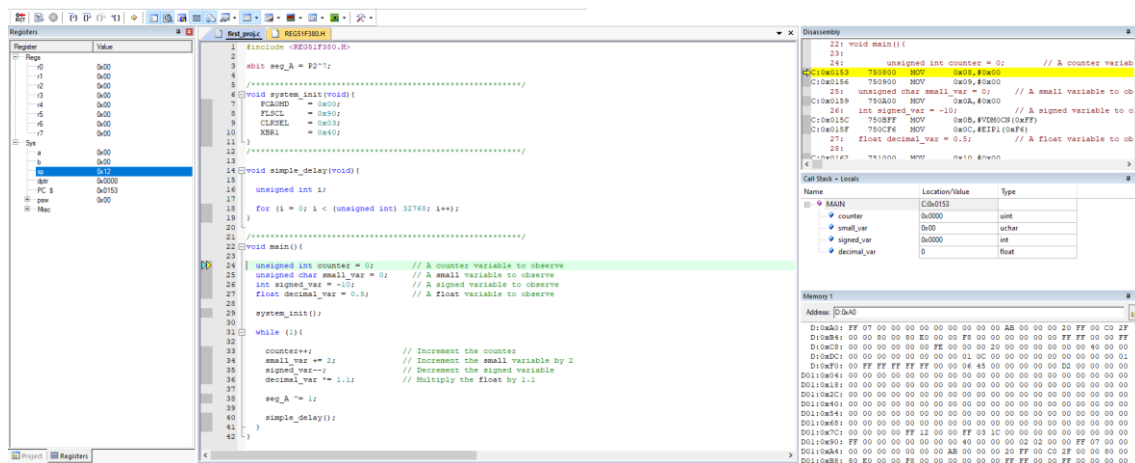
1. **Build** -faz build ao programa atual para gerar o ficheiro binário que vai ser programado no MCU
2. **Rebuild** – apaga o build anterior, e gera um ficheiro binário novo para programar o MCU
3. **Load** – programa o MCU e inicia a execução
4. **Start/Stop Debug Session** – inicia uma sessão de debug e coloca um breakpoint que por default é a função main();
5. **Insert/Remove breakpoint** – insere/remove breakpoints nas linhas pretendidas do Código.

Verifique na janela das mensagens que o código compilou sem erros e sem warnings.

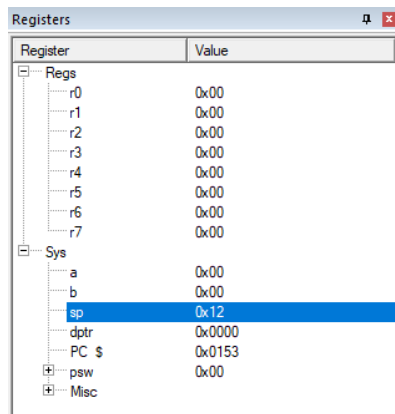
```
Build Output
Rebuild started: Project: first_project
Rebuild target 'Target 1'
compiling first_proj.c...
linking...
Program Size: data=20.0 xdata=0 code=476
creating hex file from ".\Objects\first_project"...
".\Objects\first_project" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:01
```

Inicie uma sessão de debug no botão Start/Stop debug session, onde após o código ser enviado para a memória do microcontrolador, novas janelas, funcionalidades e ferramentas aparecem. Configure o ambiente arrastando e movendo as janelas como achar conveniente.

Se tudo correr normalmente e se configurado no projeto e definições do target, o programa corre até à função principal main();



Janela **Registers** mostra o conteúdo atual dos registros do 8051



A janela de código mostra um cursor que indica a posição atual do programa

```

22: /*****
23: void main() {
24:
25:     unsigned int counter = 0;    // A counter variable to observe
26:     unsigned char small_var = 0; // A small variable to observe
27:     int signed_var = -10;        // A signed variable to observe
28:     float decimal_var = 0.5;     // A float variable to observe
29:
30:     system_init();
31:
32:     while (1){

```

A janela Disassembly mostra o código assembly gerado, com as instruções definidas no ISA do 8051.

A janela Call Stack + Locals mostra as variáveis e o valor das mesmas, declaradas no scope da função atual, neste caso a main().

A janela Memory 1 (podem ser adicionadas mais se necessário) mostra a memória a ser monitorizada. Os endereços da memória de código iniciam com o prefixo C:, memória de dados com o prefixo D:, e a memória externa com o prefixo X:.

The screenshot shows three debugger windows. The 'Disassembly' window displays the assembly code for the 'main' function, with instructions for initializing variables. The 'Call Stack + Locals' window shows the current function 'MAIN' and its local variables: 'counter' (uint, 0x0000), 'small\_var' (uchar, 0x00), 'signed\_var' (int, 0x0000), and 'decimal\_var' (float, 0). The 'Memory 1' window shows a memory dump starting at address D:0xA0, displaying hexadecimal values and their corresponding ASCII characters.

| Name        | Location/Value | Type  |
|-------------|----------------|-------|
| MAIN        | C:0x0153       |       |
| counter     | 0x0000         | uint  |
| small_var   | 0x00           | uchar |
| signed_var  | 0x0000         | int   |
| decimal_var | 0              | float |

| Address  | Value                                           |
|----------|-------------------------------------------------|
| D:0xA0   | FF 07 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |
| D:0xA4   | 00 00 80 00 00 00 E0 00 00 F8 00 00 00 00 00 00 |
| D:0xC8   | 00 00 00 00 00 00 FE 00 00 00 20 00 00 00 00 00 |
| D:0xDC   | 00 00 00 00 00 00 00 00 00 01 0C 00 00 00 00 00 |
| D:0xF0   | 00 FF FF FF FF FF 00 00 06 45 00 00 00 00 00    |
| D01:0x04 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    |
| D01:0x18 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    |
| D01:0x2C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    |
| D01:0x40 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    |
| D01:0x54 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    |
| D01:0x68 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    |
| D01:0x7C | 00 00 00 00 FF 12 00 00 FF 03 1C 00 00 00 00 00 |
| D01:0x90 | FF 00 00 00 00 00 00 00 40 00 00 00 02 02 00    |
| D01:0xA4 | 00 00 00 00 00 00 00 00 AB 00 00 00 20 FF 00 C0 |
| D01:0xB8 | 80 E0 00 00 F8 00 00 00 00 00 00 FF FF 00 00 00 |

A janela Watch mostra variáveis a monitorizar, fora do scope da função atualmente a ser executada.

| Name               | Value                 | Type             |
|--------------------|-----------------------|------------------|
| msg                | D:0x11 msg[] "LETI\n" | array[6] of char |
| decimal_var        | 0                     | float            |
| seg_A              | 1                     | bit              |
| <Enter expression> |                       |                  |

Uma sessão de debug permite parar/executar o código passo a passo, definir breakpoints (executar o código até este parar numa linha específica (breakpoint), visualizar e alterar variáveis, etc. Após iniciada, novos botões e funcionalidades aparecem.



1. Run – Executa o código normalmente até ao próximo breakpoint
2. Step – Permite entrar na função a ser chamada
3. Step Over – salta a função a ser chamada (a função executou normalmente)
4. Step Out – Executa até sair da função atual
5. Executa até à posição do cursor

### 3. Questões

- a) Diga o que faz a função **system\_init()**, explicando os valores atribuídos aos registos **PCA0MD**, **FLSCL**, **CLKSEL**, e **XBR1**. Deve verificar o reference manual ou datasheet do microcontrolador para poder responder.
- b) O que fazem estes registos e em que endereços da memória estão mapeados? Monitorize e veja se alteram quando faz a atribuição dos valores. Execute o código passo a passo, até entrar na função **system\_init()**, para verificar o efeito das alterações.
- c) O que faz a linha `sbit seg_A = P2^7`? O que faz, e em que memória e endereço está mapeado o registo P2? Monitorize para ver as alterações.
- d) O que faz a linha `seg_A ^= 1`; e qual o seu efeito na placa? Porquê?
- e) Adicione uma janela Watch1 à sua configuração. Esta janela permite visualizar e monitorizar variáveis que não estão no scope da função atualmente a ser executada. Adicione o array `msg[]` à janela e `seg_A` para ver os valores a serem alterados.
- f) Em que memória e posição estão guardados os dados do array `msg1[]`, `msg[2]`, `msg[3]`, `msg[4]`, `msg[5]`, e `msg[6]`? Verifique na janela Memory1. Adicione mais janelas de memory se achar necessário.
- g) O que fazem as diretivas `data`, `code`, `xdata`, e `_at_`?
- h) Verifique e entenda o código das linhas 33 a 36. O que faz esse código? Mostre onde ficou guardado o resultado final da operação.
- i) Tente realizar a mesma operação, mas copiando `msg4[]` para `msg[2]`. Diga o que aconteceu.
- j) Coloque um breakpoint na linha 39, `seg_A ^= 1`. Para isso pode fazer click do lado esquerdo e um ponto vermelho vai aparecer. Execute o código até esta linha.

```

22  /*****
23  void main() {
24
25      unsigned int counter = 0;          // A counter variable to observe
26      unsigned char small_var = 0;      // A small variable to observe
27      int signed_var = -10;             // A signed variable to observe
28      float decimal_var = 0.5;          // A float variable to observe
29
30      system_init();
31
32      while (1) {
33
34          seg_A ^= 1;
35
36          counter++;                    // Increment the counter
37          small_var += 2;               // Increment the small variable by 2
38          signed_var--;                // Decrement the signed variable
39          decimal_var *= 1.1;           // Multiply the float by 1.1
40
41          simple_delay();
42      }
43  }

```

- k) Que valor tem neste momento as variáveis seg\_A, small\_var, decimal\_var, signed\_var, e counter? Em que memória estão estas variáveis armazenadas?
- l) Quando deixam estas variáveis de ser atualizadas e porquê? Qual o seu valor final?
- m) O que faz infinitamente o código?

#### 4. Elementos a entregar

Ficheiro txt com as respostas às questões apresentadas. Prints de janelas do ambiente de desenvolvimento keil se achar necessário.